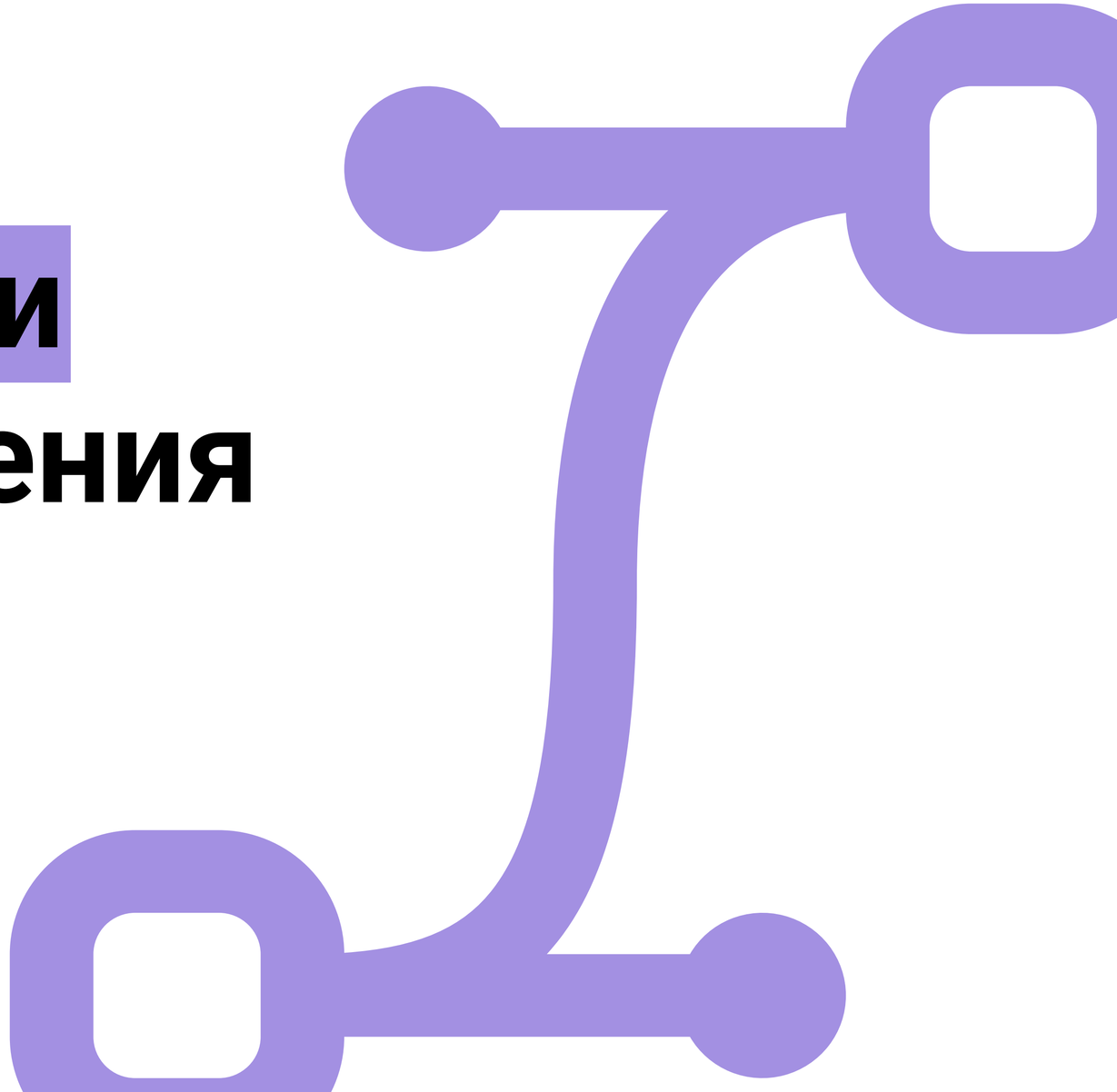
2026

Платформы и среды выполнения ЯП

NEXIGN x ИТМО

Раздел 02

Ландшафт памяти и модели исполнения



На предыдущем занятии:

Кто и на каком основании решает, что объект больше не нужен?

На этом занятии:

Как определить, какие объекты действительно больше не нужны программе?



Подсчёт ссылок vs. достижимость

Критерий	Подсчёт ссылок	Трассировка
Критерий жизни	Счётчик ссылок больше нуля	Достижимость от корней
Циклические ссылки	Требуют дополнительных механизмов	Обрабатываются естественным образом
Накладные расходы	Обновление счётчиков	Обход графа объектов
Время освобождения	При обнулении счётчика	Во время сборки
Примеры	CPython, Swift ARC	JVM, CLR, V8, Go

Куча как граф объектов

- Объекты — вершины графа.
- Ссылки — направленные рёбра.
- Корни — начальные вершины обхода.

GC Roots

- Локальные переменные и временные ссылки в стеках потоков.
- Глобальные и статические переменные.
- Регистры и сохранённое состояние потоков.
- Внутренние структуры среды выполнения.
- Специальные ссылки, например JNI handles в JVM.

Алгоритм трассировки: наивный

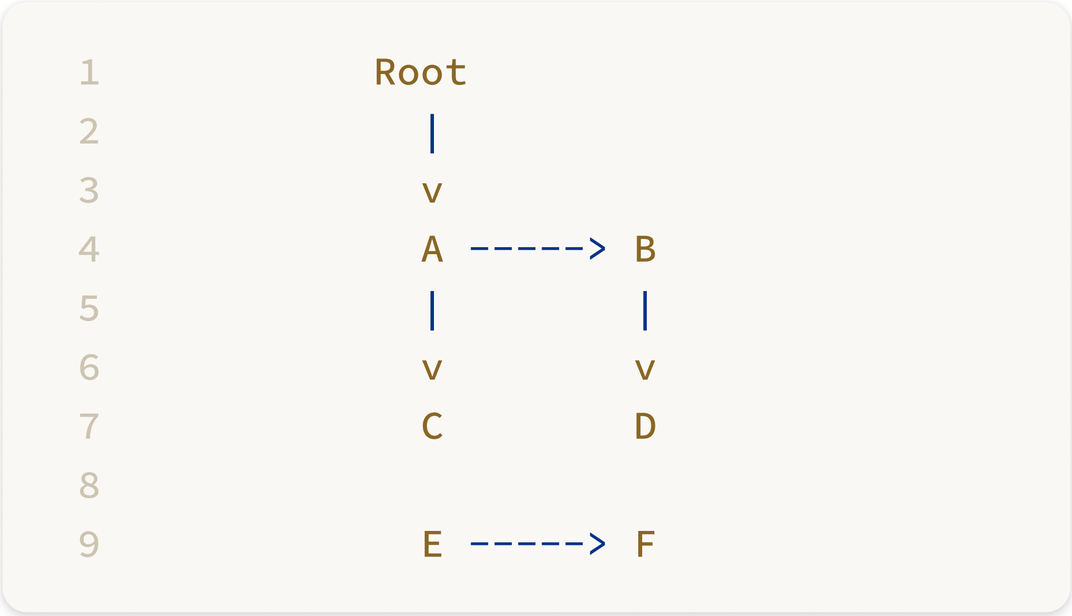
```
1  mark(root):
2      if root is null or root.marked:
3          return
4
5      root.marked = true
6
7      for each reference in root.references:
8          mark(reference)
```

- Начинаем с корней.
- Обходим все ссылки.
- Каждый объект посещается не более одного раза.
- Достижимые объекты помечаются как живые.
- Непомеченные объекты могут быть освобождены.

Mark-and-Sweep

Две фазы:

1. Mark — маркировка достижимых объектов.
2. Sweep — обход кучи и освобождение непомеченных объектов.




```
До GC:
[A] [B] [C] [D] [E] [F]

После Mark:
[A*] [B*] [C] [D*] [E] [F*]

После Sweep:
[A] [B] [FREE] [D] [FREE] [F]
```

Mark-and-Sweep

Плюсы:

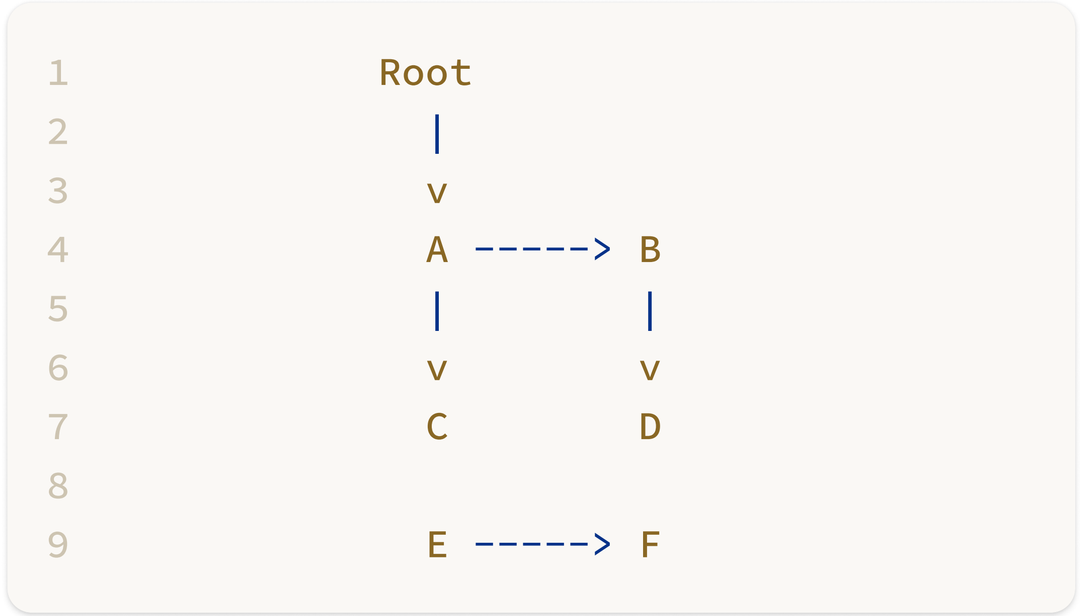
- Относительно простая реализация.
- Не требуется перемещение объектов.
- Подходит для объектов разного размера.

Минусы:

- Внешняя фрагментация.
- Необходимость обхода области кучи при sweep.
- Возможные продолжительные STW (stop-the-world) паузы.

Mark-and-Compact

Расширение Mark-and-Sweep: после маркировки живые объекты перемещаются в непрерывную область памяти.



1. Mark — маркировка живых объектов.
2. Plan — вычисление новых адресов.
3. Compact — перемещение объектов и обновление ссылок.

```
До:
[A*] [B] [C*] [D] [E*] [F]

После:
[A] [C] [E] [FREE] [FREE] [FREE]
```

Copying GC

```
До GC:

From-Space          To-Space
[A] [B] [C] [D]     [EMPTY]

После GC:

From-Space          To-Space
[EMPTY]             [A] [C] [FREE]
```

- Куча делится на две области.
- Живые объекты копируются в другую область.
- Ссылки обновляются.
- Старая область освобождается целиком.

Преимущества:

- Отсутствие фрагментации.
- Простая аллокация через bump-pointer.
- Быстрое освобождение старой области.

Недостатки:

- Необходимость дополнительной памяти.
- Стоимость копирования живых объектов.
- Неэффективность при высокой доле долгоживущих объектов. на две области.

Сравнение классических алгоритмов

Критерий	**Mark-and-Sweep**	**Mark-and-Compact**	**Copying GC**
Перемещение объектов	Нет	Да	Да
Фрагментация	Возможна	Устраняется	Устраняется
Дополнительная память	Небольшая	Зависит от реализации	Значительная
Освобождение памяти	По результатам sweep	Перемещение и очистка	Очистка всей области
Стоимость	Обход + sweep	Обход + перемещение	Обход + копирование живых объектов

Гипотеза о поколениях

- Большинство объектов умирает молодыми.
- Объекты, “пережившие” несколько сборок, с большей вероятностью проживут долго.

```
Создание объекта
        |
        v
Young Generation
        |
        | пережил GC
        v
Intermediate / Old
        |
        | пережил несколько GC
        v
Long-lived object
```

Generational GC

- Новые объекты выделяются в Young Generation.
- Молодое поколение собирается чаще (Minor GC).
- Выжившие объекты продвигаются в старшее поколение (promotion).
- Старшее поколение собирается реже (Major GC).
- Full GC может затрагивать всю кучу.
- Конкретное количество поколений и правила promotion зависят от реализации.

Проблема межпоколенческих ссылок

- Объект старшего поколения может ссылаться на молодой.
- При Minor GC нельзя игнорировать такие ссылки.
- Полное сканирование старшего поколения при каждой сборке уничтожило бы преимущества поколенческой модели.



Проблема межпоколенческих ссылок

Решение:

- Write Barrier — отслеживание изменений ссылок.
- Remembered Set — хранение информации о ссылках между областями или поколениями.
- При Minor GC учитываются корни и зарегистрированные межпоколенческие ссылки.

Write barrier

```
// Что пишет программист:
obj.field = newValue
// Что генерирует компилятор (псевдокод):
writeBarrier(obj, &obj.field, newValue)
obj.field = newValue
```

Stop-the-World

Почему GC останавливает приложение:

- Для корректной трассировки требуется согласованное состояние графа объектов.
- При классическом STW все управляемые потоки приостанавливаются.
- Сборщик получает возможность безопасно обходить ссылки.

Проблемы:

- Длительные паузы.
- Увеличение задержек ответа.
- Непредсказуемость времени выполнения интерактивных приложений.

Parallel GC

- Несколько потоков сборщика одновременно выполняют работу.
- Сокращается время выполнения сборки за счёт нескольких ядер.
- Потоки приложения при этом могут оставаться приостановленными.
- Основная цель — увеличение throughput.

Concurrent GC

- GC выполняет часть работы одновременно с приложением.
- Потоки приложения продолжают исполнять код.
- Необходимо отслеживать изменения графа объектов.
- Дополнительные расходы на синхронизацию и CPU.

Проблема конкурентной маркировки

```
До изменения:

Root ---> A ---> B

GC уже обработал A.

Приложение изменяет граф:

Root ---> A
          |
          +----> C
```

- GC уже завершил обработку объекта A.
- Приложение добавило новую ссылку A → C.
- Без дополнительных механизмов C может остаться непомеченным.
- Сборщик ошибочно освободит живой объект.

Решение через write barriers:

- Отслеживание изменений графа.
- Регистрация новых ссылок.
- Сохранение корректности маркировки при конкурентном выполнении.

Incremental GC

- Сборка разбивается на небольшие порции.
- Между порциями приложение продолжает работу.
- Снижается длительность отдельных пауз.
- Усложняется управление состоянием графа.

Сравнение подходов

Подход	Основная идея	Цель
STW	Приостановить приложение	Согласованность
Parallel	Параллельная работа GC-потоков	Throughput
Concurrent	Одновременная работа GC и приложения	Latency
Incremental	Разделить работу на порции	Ограничение пауз

Java Virtual Machine: G1, ZGC, Shenandoah

Сборщик	Архитектура	Цель
G1	Поколенческий, региональный, частично конкурентный: делит кучу на регионы и выбирает наиболее выгодные области для сборки.	Баланс throughput и latency
ZGC	Конкурентная сборка и перемещение: использует механизмы, позволяющие выполнять значительную часть работы конкурентно.	Минимизация пауз
Shenandoah	Конкурентная маркировка и перемещение: стремится минимизировать время STW за счёт конкурентного перемещения объектов.	Минимизация пауз

.NET CLR: поколения и Large Object Heap

- Workstation GC — для клиентских приложений.
- Server GC — для серверных сценариев.
- Background GC — сборка с уменьшением времени приостановки приложения.

```
Managed Heap

Generation 0
     |
     v
Generation 1
     |
     v
Generation 2

Large Object Heap (LOH)
```

JavaScript V8: Orinoco

- Молодое поколение: Scavenger (копирующая сборка).
- Старшее поколение: Mark-Sweep-Compact.
- Incremental Marking.
- Concurrent Marking и Sweeping.
- Parallel Scavenging.

Основная идея: быстро собирать многочисленные краткоживущие объекты и не останавливать JavaScript-приложение на длительное время при работе со старшим поколением.

Go Runtime: конкурентный Mark-and-Sweep

- Конкурентная маркировка достижимых объектов.
- Использование write barriers.
- Pacing — управление моментом запуска и интенсивностью сборки.
- Нет традиционного деления кучи на поколения в стандартной реализации GC.

Python (CPython): гибридная модель

- Подсчёт ссылок — основной механизм управления временем жизни объектов.
- Циклический сборщик обнаруживает определённые недостижимые циклические структуры.
- Сборка циклов дополняет reference counting, а не заменяет его.

Как измерять эффективность GC

Метрика	Что измеряет
Throughput	Долю времени полезной работы приложения
Pause Time	Продолжительность пауз GC
Latency	Время ответа приложения, включая хвостовые задержки
Heap Size	Объём управляемой кучи
Allocation Rate	Скорость выделения объектов
GC CPU Overhead	Затраты CPU на сборку мусора

Итого

1. Трассирующая сборка определяет живые объекты по достижимости из GC Roots.
2. Циклические ссылки не мешают трассирующей сборке обнаруживать недостижимые объекты.
3. Mark-and-Sweep, Mark-and-Compact и Copying GC по-разному организуют освобождение памяти.
4. Generational GC использует гипотезу о коротком времени жизни большинства объектов.
5. Remembered Sets и Write Barriers позволяют эффективно учитывать изменения графа ссылок.
6. Parallel, Concurrent и Incremental GC решают разные задачи оптимизации времени сборки и пауз.
7. Промышленные среды выполнения используют разные комбинации алгоритмов в зависимости от требований к throughput, latency и потреблению памяти.
8. Автоматическое управление памятью не исключает утечек: достижимый, но больше не нужный объект может продолжать занимать память.